

Overview

Part 99 — 🎓 Capstone Projects · 99-capstones/README.md

🎓 Capstone Projects

Three projects, each pulling together a specific slice of the track. They are deliberately scoped to be buildable in a weekend to a week by one person with the modules below already done — not a research program. Pick the one closest to what you actually want to be able to say you've built.

Each project lists **prerequisite modules**, a **spec**, an **explicit non-goal list** (what you are allowed to skip so the project stays finishable), and an **evaluation bar** — a concrete, checkable definition of done, in the spirit of every module's "exit check."

Capstone A — A grounded RAG assistant over your own documents

Prerequisites: 01 [01-ml-foundations], 03 [03-nlp], 04 [04-llm], 08 [08-rag], and one of 16.1 Python [16-ai-tech-stack/tracks/python] / 16.2 TypeScript [16-ai-tech-stack/tracks/typescript].

Spec: ingest a real document set (your own notes, a product's docs, a set of PDFs — 50–500 pages is plenty), chunk it, embed it, index it, and answer questions with **inline citations back to source chunks**. Include a query rewriting step and a reranking pass. Abstain explicitly (`INSUFFICIENT_CONTEXT`, see 04's prompt patterns [04-llm/code/prompt_patterns.md]) when the retrieved context doesn't support an answer.

Non-goals: no need for a UI beyond a CLI or a single script; no need for a managed vector DB — in-memory or SQLite is fine at this scale; no need for multi-turn memory.

Evaluation bar: build a 20-question eval set by hand (half answerable from your corpus, half deliberately not). Report recall@5 for retrieval and a faithfulness score for generation (Module 08's eval metrics [08-rag], Module 15 [15-ai-evals]). The abstention rate on the unanswerable half should be high; the citation accuracy on the answerable half should be checkable by you by hand in under two minutes per question.

Capstone B — A tool-using agent with a real approval gate

Prerequisites: 04 [04-llm], 06 [06-ai-agents], 09 [09-mcp], 16.4 LangGraph [16-ai-tech-stack/tracks/langgraph], and skim 13 [13-ai-security].

Spec: build an agent that can take at least one **consequential, hard-to-reverse action** (send an email, create a calendar event, modify a file outside a sandbox, place an order against a fake/test API) gated behind a genuine human-in-the-loop approval step — not a static breakpoint, an `interrupt()`-style pause that resumes correctly after a real delay. Give it at least 3 tools, one of which can fail, and handle the failure without crashing the loop. Wire in at least one MCP server (your own minimal one from Module 09 [09-mcp/code] is fine).

Non-goals: no need for multi-agent orchestration — one agent is the point of this capstone (Module 07 [07-agentic-ai] makes the case for why that's usually the right call anyway); no need for a production deployment.

Evaluation bar: run it end to end with the human saying "reject" once and "approve" once. Show that state resumed correctly both times. Deliberately feed it a tool result containing an injected instruction (see Module 13's demo [13-ai-security]) and show your defense catching it.

Capstone C — An evaluated, governed feature, not just a working demo

Prerequisites: any one working system from Capstone A or B, plus 12 [12-ai-governance], 14 [14-ai-compliance], 15 [15-ai-evals], and 16.5 LangSmith [16-ai-tech-stack/tracks/langsmith] or Module 15's own tools [15-ai-evals/code].

Spec: take whatever you built in A or B and wrap it in the artifacts a real organization would require before shipping it: a filled-out model/system card (Module 12's template [12-ai-governance/code/model_card_template.md]), a risk-tier classification with stated reasoning (Module 12 [12-ai-governance/code/ai_risk_register_template.py] and, if relevant, Module 14's EU AI Act classifier [14-ai-compliance/code/eu_ai_act_risk_classifier.py]), a CI-style eval gate that fails the build on a quality or safety regression (Module 15 [15-ai-evals/code/statistical_rigor_for_evals.py]), and full request tracing.

Non-goals: you are not seeking real legal sign-off — this is the practice of producing the artifacts, not a compliance filing. Say so explicitly in your write-up.

Evaluation bar: hand a colleague (or a future version of yourself, in a month) just the artifacts — no verbal explanation. They should be able to answer "what does this do, what's it not allowed to do, how do we know it works, and what happens when it's wrong" from the documents alone.

After you finish one

Write down, in three sentences: what surprised you, what took far longer than expected, and which module you had to go back and re-read. That note is worth more than the code — it's the beginning of your own "failure taxonomy" (Module 15 [15-ai-evals]), and it's exactly what a real team's retrospective looks like.